

CyberArk Practice Tracker — Phased Development Plan

Audience: Frontend developer (you) + Node.js/MySQL backend developer

Goal: Ship a working MVP first, then layer in features incrementally

HOW TO READ THIS DOCUMENT

Each phase has two sections:

- **Frontend tasks** — what you do in HTML/JS/CSS
- **Backend briefing** — exactly what to ask your Node.js developer to build so you can connect to it

The backend section is written so you can copy-paste it directly to your developer.

PHASE 0 — Foundation (Backend-first, No UI yet)

Goal: Get a real database behind the app so data is not lost on refresh. Nothing new is visible to the user — the existing app just starts reading from and writing to the API instead of in-memory arrays.

Who does what: Backend does all the heavy lifting here. Frontend makes the switch from in-memory to API calls.

Phase 0 — Backend Briefing (copy this to your developer)

What we are building

We are replacing the in-memory JavaScript arrays in our single-page app with a Node.js REST API backed by MySQL. The frontend is currently a single HTML file. All data is in JS variables. We need to persist that data in MySQL and expose it through REST endpoints.

Tech stack expected

- Node.js with Express

- MySQL (via `mysql2` package recommended)
 - No auth for now (MVP — all endpoints are open)
 - JSON request and response bodies throughout
 - Standard HTTP status codes (200 OK, 201 Created, 400 Bad Request, 404 Not Found, 500 Server Error)
-

Database schema — Phase 0

Please create these tables. Use `utf8mb4` charset throughout.

```
sql
```

```

-- Team members (the people who can own projects or be assigned tasks)
CREATE TABLE team_members (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  name        VARCHAR(100) NOT NULL,
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Customers (organisations we work with)
CREATE TABLE customers (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  name        VARCHAR(200) NOT NULL UNIQUE,
  industry    VARCHAR(100),
  cyberark_tenant VARCHAR(200),
  region      VARCHAR(100),
  idp         VARCHAR(100),
  siem        VARCHAR(100),
  license_type VARCHAR(100),
  license_count INT,
  license_expiry DATE,
  notes       TEXT,
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Projects
CREATE TABLE projects (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  customer_id INT NOT NULL,
  type        ENUM('Implementation', 'Managed Service', 'License Renewal', 'New Opportunity'),
  owner_id    INT, -- FK to team_members
  status      ENUM('On Track', 'At Risk', 'Delayed', 'Completed', 'Prospecting') NOT NULL,
  start_date  DATE,
  due_date    DATE,
  notes       TEXT,
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (customer_id) REFERENCES customers(id) ON DELETE CASCADE,
  FOREIGN KEY (owner_id)    REFERENCES team_members(id) ON DELETE SET NULL
);

-- Activity groups (the phases inside a project, e.g. "Vault Configuration")
CREATE TABLE activity_groups (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  project_id  INT NOT NULL,
  name        VARCHAR(200) NOT NULL,

```

```

position      INT NOT NULL DEFAULT 0,      -- sort order within the project
created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
FOREIGN KEY (project_id) REFERENCES projects(id) ON DELETE CASCADE
);

-- Sub-tasks (the individual steps inside an activity group)
CREATE TABLE subtasks (
  id           INT AUTO_INCREMENT PRIMARY KEY,
  group_id     INT NOT NULL,
  name         VARCHAR(300) NOT NULL,
  assignee_id  INT,                        -- FK to team_members, nullable
  status       ENUM('Not Started','In Progress','In Testing','Awaiting Feedback'),
  due_date     DATE,
  position     INT NOT NULL DEFAULT 0,    -- sort order within the group
  flag_type    VARCHAR(100),              -- NULL means not flagged
  flag_reason  TEXT,
  flag_waiting_on VARCHAR(200),
  created_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (group_id) REFERENCES activity_groups(id) ON DELETE CASCADE,
  FOREIGN KEY (assignee_id) REFERENCES team_members(id) ON DELETE SET NULL
);

-- Contacts (people at the customer organisation)
CREATE TABLE contacts (
  id           INT AUTO_INCREMENT PRIMARY KEY,
  customer_id  INT NOT NULL,
  name         VARCHAR(200) NOT NULL,
  role         VARCHAR(100),
  department   VARCHAR(100),
  email        VARCHAR(200),
  phone        VARCHAR(50),
  notes        TEXT,
  created_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (customer_id) REFERENCES customers(id) ON DELETE CASCADE
);

-- Documents linked to a customer
CREATE TABLE documents (
  id           INT AUTO_INCREMENT PRIMARY KEY,
  customer_id  INT NOT NULL,
  name         VARCHAR(300) NOT NULL,
  type         ENUM('HLD','LLD','SOP','Architecture Diagram','Network Diagram','Infra
status        ENUM('Draft','In Review','Approved','Shared') NOT NULL DEFAULT 'Draft'
link          VARCHAR(500),

```

```

    notes          TEXT,
    created_at     TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (customer_id) REFERENCES customers(id) ON DELETE CASCADE
);

-- Infrastructure / servers linked to a customer
CREATE TABLE infra_servers (
    id              INT AUTO_INCREMENT PRIMARY KEY,
    customer_id     INT NOT NULL,
    hostname        VARCHAR(200) NOT NULL,
    ip_address      VARCHAR(50),
    os              VARCHAR(100),
    role            ENUM('PVWA', 'CPM', 'PSM', 'DR Vault', 'Primary Vault', 'Jump Server', 'Other'),
    environment     ENUM('Production', 'UAT', 'Dev', 'POC') NOT NULL DEFAULT 'Production',
    notes          TEXT,
    created_at     TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (customer_id) REFERENCES customers(id) ON DELETE CASCADE
);

```

Seed data script

Please provide a seed SQL script or a `/api/seed` endpoint (development only, not exposed in production) that inserts the 11 demo projects, their activity groups, and sub-tasks from the original HTML file. The frontend will call this once on first setup to populate the database.

REST API — Phase 0 endpoints

Please implement these endpoints. All responses are JSON. All list endpoints return arrays. All single-record endpoints return the object or 404.

Team Members

GET	/api/team	→ list all team members
POST	/api/team	→ create { name }
DELETE	/api/team/:id	→ delete (return 409 if member is assigned
	to any open subtask)	

Customers

GET	/api/customers	→ list all (id, name, industry, license_type, license_expiry)
GET	/api/customers/:id	→ full customer object including profile fields
POST	/api/customers	→ create { name, ...profile fields }
PUT	/api/customers/:id	→ update profile fields

Projects

GET	/api/projects	→ list all projects each item includes: id, customer_id, customer_name, type, owner_id, owner_name, status, start_date, due_date, notes, progress (computed: done_count/total_count)
GET	/api/projects/:id	→ single project with full task tree (groups → subtasks)
POST	/api/projects	→ create { customer_id, type, owner_id, status, start_date, due_date, notes } auto-insert activity groups + subtasks from the template for that type
PUT	/api/projects/:id	→ update { owner_id, status, start_date, due_date, notes }
DELETE	/api/projects/:id	→ delete project and cascade

Template logic: the backend should have the same 4 project-type templates (Implementation / Managed Service / License Renewal / New Opportunity) hardcoded as seed data. When a project is created, the matching template's groups and subtasks are inserted automatically. The frontend should not have to send the task list on creation.

Activity Groups

POST	/api/projects/:pid/groups	→ create { name, position }
PUT	/api/groups/:id	→ update { name, position }
DELETE	/api/groups/:id	→ delete group and all its subtasks

Sub-tasks

Contacts

```
GET    /api/customers/:id/contacts    → list contacts for customer
POST   /api/customers/:id/contacts    → create { name, role, department, email,
phone, notes }
DELETE /api/contacts/:id              → delete contact
```

Documents

```
GET    /api/customers/:id/documents    → list documents for customer
POST   /api/customers/:id/documents    → create { name, type, status, link, notes
}
DELETE /api/documents/:id             → delete document
```

Infra Servers

```
GET    /api/customers/:id/infra        → list servers for customer
POST   /api/customers/:id/infra        → create { hostname, ip_address, os, role,
environment, notes }
DELETE /api/infra/:id                 → delete server
```

Dashboard summary (computed)

```
GET    /api/dashboard                  → return:
{
  total_projects: number,
  by_type: { Implementation: n, "Managed Service": n, ... },
  needs_attention: [ { id, customer_name, type, owner_name, status,
blocked_count } ],
  due_in_30_days: [ { id, customer_name, type, due_date } ]
}
```

CORS

The frontend will be served from a different origin during development. Please enable CORS for `localhost` in dev and lock it to the production domain later.

Response conventions

- Dates: return as `YYYY-MM-DD` strings (not full ISO timestamps)
 - IDs: always integers
 - Null fields: return as `null` not omitted
 - Errors: return `{ error: "human readable message" }` with the appropriate HTTP status
-

Phase 0 — Frontend Tasks

Once the backend endpoints are ready:

#	Task	Notes
F0.1	Create an <code>api.js</code> module (or inline functions) with wrapper functions for every endpoint	e.g. <code>getProjects()</code> , <code>createProject(data)</code> , <code>updateSubtask(id, data)</code> — all return promises
F0.2	Replace <code>projects</code> in-memory array with <code>await getProjects()</code> call on page load	Show a loading state while fetching
F0.3	Replace <code>customers</code> in-memory object with <code>await getCustomers()</code> call	
F0.4	Replace <code>teamMembers</code> array with <code>await getTeam()</code> call	
F0.5	Wire <code>saveProject()</code> (modal save) to <code>POST /api/projects</code> or <code>PUT /api/projects/:id</code>	
F0.6	Wire <code>toggleSub()</code> (checkbox click) to <code>PUT /api/subtasks/:id</code> with <code>{ status }</code>	
F0.7	Wire <code>setSubDue()</code> (date picker change) to <code>PUT /api/subtasks/:id</code> with <code>{ due_date }</code>	
F0.8	Wire all contact / document / infra save and delete actions to their endpoints	
F0.9	Replace the dashboard computed values with data from <code>GET /api/dashboard</code>	
F0.10	Add a global error handler — if any API call fails, show a small error toast at the bottom of the screen	

Definition of done for Phase 0: Refresh the page. All data still shows. No data is lost.

PHASE 1 — Sub-task Status + Flag (MVP Core)

Goal: The two features that answer "why is this project delayed?" — status per step and the block/flag mechanism. This is the highest-value change and justifies the whole rewrite.

Phase 1 — Backend Briefing

The schema already has `status`, `flag_type`, `flag_reason`, `flag_waiting_on` columns in the `subtasks` table from Phase 0. No new tables needed.

One new endpoint needed:

```
GET /api/my-tasks?member_id=:id    → all subtasks assigned to a team member
                                     return:
                                     [
                                     {
                                     subtask_id, subtask_name, status,
due_date,
                                     flag_type, flag_reason,
                                     group_id, group_name,
                                     project_id, customer_name
                                     }
                                     ]
                                     ordered by:
                                     1. overdue + not Done (due_date <
today, status != 'Done')
                                     2. due within 7 days + not Done
                                     3. not Done, no due date
                                     4. Done
```

Update to `/api/dashboard`:

Add `blocked_count` per project in the `needs_attention` list. This is a count of subtasks where `flag_type IS NOT NULL` OR `status IN ('Blocked', 'Awaiting Feedback')`.

Also include projects in `needs_attention` even if their project-level status is "On Track", if they have blocked subtasks.

1A — Status badge on sub-task row

#	Task	Notes
F1.1	Add a <code>status</code> column to the sub-task row CSS grid	New layout: <code>[name - flex] [date - 120px] [status - 110px] [checkbox - 30px]</code>
F1.2	Render the status as a coloured badge using existing <code>.badge</code> classes	Map: Not Started→gray, In Progress→blue, In Testing→teal, Awaiting Feedback→amber, Blocked→red, Done→green
F1.3	Make the badge clickable — clicking opens a small inline status picker dropdown	List all 6 statuses; close on outside click
F1.4	On status pick, call <code>PUT /api/subtasks/:id</code> with <code>{ status }</code> then re-render	If status = "Done", also set checkbox to checked; if anything else, uncheck
F1.5	Update <code>toggleSub()</code> (checkbox) to set status = "Done" when checked, "In Progress" when unchecked	Calls the same <code>PUT /api/subtasks/:id</code> endpoint
F1.6	Update <code>taskProg()</code> to count subtasks where <code>status === "Done"</code> instead of <code>done === true</code>	

1B — Flag modal

#	Task	Notes
F1.7	Add a flag icon () to each sub-task row	Gray/faint = no flag; red/filled = flagged
F1.8	Add flag modal HTML to the page	Fields: flag type dropdown, reason textarea, waiting-on input, Save / Clear flag / Cancel buttons
F1.9	<code>openFlagModal(subtaskId)</code> — opens modal; pre-fills fields if flag already set; shows "Clear flag" only if flag exists	

#	Task	Notes
F1.10	<code>saveFlag()</code> — calls <code>PUT /api/subtasks/:id</code> with flag fields + sets status to "Blocked" or "Awaiting Feedback" based on flag type	Flag types that set "Awaiting Feedback": "Waiting for approval", "Waiting for customer", "Waiting for third party". All others set "Blocked".
F1.11	<code>clearFlag()</code> — calls <code>PUT /api/subtasks/:id</code> with <code>{ flag_type: null, flag_reason: null, flag_waiting_on: null, status: "In Progress" }</code>	
F1.12	Show a tooltip on the filled flag icon showing flag type + reason	Use HTML <code>title</code> attribute for simplest approach
F1.13	Add a  count to activity group headers where any subtask in that group is flagged	e.g. "  2" shown in the group header meta area

1C — Dashboard update

#	Task	Notes
F1.14	Update "Needs Attention" rendering to also show projects with blocked subtasks (even if project status is On Track)	Use <code>blocked_count</code> from the dashboard API response
F1.15	Show a small red badge on each Needs Attention row with count of blocked steps	e.g. <code>2 blocked</code>

Definition of done for Phase 1: A manager can open any project, see the status of every step at a glance, flag a blocked step with a reason, and the dashboard immediately reflects it.

PHASE 2 — CRUD for Steps and Phases

Goal: Users can add, rename, and delete steps and phases directly in the UI. Makes the tracker usable for projects that don't perfectly match the template.

Phase 2 — Backend Briefing

No new tables. All needed endpoints were already defined in Phase 0:

- `POST /api/projects/:pid/groups` — add a phase
- `PUT /api/groups/:id` — rename a phase
- `DELETE /api/groups/:id` — delete a phase + all its steps
- `POST /api/groups/:gid/subtasks` — add a step
- `PUT /api/subtasks/:id` — rename a step (same endpoint, just send `{ name }`)
- `DELETE /api/subtasks/:id` — delete a step

One clarification: `DELETE /api/groups/:id` should return a 200 with `{ deleted: true, subtasks_deleted: N }` so the frontend can show a confirmation message if needed.

Phase 2 — Frontend Tasks

2A — Sub-task CRUD

#	Task	Notes
F2.1	Add a "+ Add step" link at the bottom of each open group's subtask list	Appears as a faint text link below the last sub-task
F2.2	Clicking "+ Add step" inserts an inline input row at the bottom of that group	Input with a confirm (✓) and cancel (✕) button
F2.3	On confirm, call <code>POST /api/groups/:gid/subtasks</code> with <code>{ name, position }</code> then re-render	Validate: name must not be blank (show inline red text if blank)
F2.4	Add a pencil (✎) icon to each subtask row — visible on row hover	Clicking turns the name into an inline editable input
F2.5	On name edit confirm, call <code>PUT /api/subtasks/:id</code> with <code>{ name }</code>	Escape cancels, Enter saves
F2.6	Add a delete (✕) icon to each subtask row — visible on row hover	Clicking shows an inline confirm prompt on the same row ("Delete? Yes / No")
F2.7	On delete confirm, call <code>DELETE /api/subtasks/:id</code> then re-render	

2B — Activity group CRUD

#	Task	Notes
F2.8	Add a "+ Add phase" button in the task panel header area	Opens a small modal or inline prompt for the phase name
F2.9	On save, call <code>POST /api/projects/:pid/groups</code> with <code>{ name, position: groups.length }</code> then re-render	
F2.10	Add a pencil icon on each activity group header (visible on hover)	Clicking turns the group name into an inline editable input
F2.11	On group name edit confirm, call <code>PUT /api/groups/:id</code> with <code>{ name }</code>	
F2.12	Add a delete icon on each activity group header (visible on hover)	Show a confirmation: "Delete this phase and all N steps? Yes / Cancel"
F2.13	On phase delete confirm, call <code>DELETE /api/groups/:id</code> then re-render	

Definition of done for Phase 2: A user can fully customise a project's task list — add new steps, rename steps, delete steps, add new phases, rename phases, delete phases — and the changes persist across refresh.

PHASE 3 — Assignees and My Tasks Page

Goal: Every step has an owner. A dedicated page shows each team member's workload across all projects.

Phase 3 — Backend Briefing

No new tables. The `subtasks.assigned_id` column and `team_members` table already exist from Phase 0.

Endpoints needed (all defined in Phase 0):

- `GET /api/team` — for populating the assignee picker dropdown
- `PUT /api/subtasks/:id` with `{ assignee_id }` — for assigning/unassigning
- `GET /api/my-tasks?member_id=:id` — for the My Tasks page (defined in Phase 1)
- `POST /api/team` and `DELETE /api/team/:id` — for team management

One clarification on `DELETE /api/team/:id`:

If the member is the `assignee_id` on any subtask where status is NOT "Done", return HTTP 409 with:

```
json
{ "error": "This person is assigned to N open tasks. Reassign them before removing." }
```

Phase 3 — Frontend Tasks

3A — Assignee chip on sub-task row

#	Task	Notes
F3.1	Add an assignee column to the sub-task row grid	New layout: <code>[name - flex] [assignee - 90px] [date - 120px] [status - 110px] [checkbox - 30px] [icons - 54px]</code>
F3.2	Render assignee as a coloured initials avatar chip (same style as project owner avatars)	If no assignee, show a faint "+" circle as a placeholder
F3.3	Clicking the chip opens a small dropdown listing all team members + "Unassign" at the bottom	Close on outside click
F3.4	On selection, call <code>PUT /api/subtasks/:id</code> with <code>{ assignee_id }</code> then re-render	Unassign sends <code>{ assignee_id: null }</code>

3B — My Tasks page

#	Task	Notes
F3.5	Add "My Tasks" nav item in the sidebar (below Customers)	
F3.6	Add <code>currentPage === "myTasks"</code> branch in <code>showPage()</code> and <code>render()</code>	
F3.7	Build <code>renderMyTasks()</code> — calls <code>GET /api/my-tasks?member_id=:id</code> and renders the result	
F3.8	At the top of My Tasks: person picker control (tab strip for ≤6 people, dropdown if more)	Stores selected member ID in <code>selectedMember</code> state variable
F3.9	Render task list grouped by project, then by activity group	Each item: group name, step name, status badge, due date,  if flagged
F3.10	Sort within each project section: overdue first (red name text), then due within 7 days (amber), then no date, then Done	
F3.11	Clicking a project name navigates to Projects page with that project row expanded	Set <code>expandedId = project_id</code> before calling <code>showPage("projects")</code>
F3.12	Empty state: "No tasks assigned to [name] yet."	

3C — Team management

#	Task	Notes
F3.13	Add a ⚙ gear icon in the sidebar footer	Opens team management modal
F3.14	Build team management modal: list current members, "+ Add member" input, remove button per member	
F3.15	" + Add member": call <code>POST /api/team</code> with <code>{ name }</code> , refresh team list	

#	Task	Notes
F3.16	Remove member: call <code>DELETE /api/team/:id</code> ; if 409 returned, show the error message from the API instead of removing	
F3.17	After any team change, refresh the assignee dropdowns in the currently open task panel	Re-call <code>getTeam()</code> and store in the module-level <code>teamMembers</code> variable

Definition of done for Phase 3: Every step has an assignee. Any team member can open My Tasks and see exactly what is waiting for them, across all projects, sorted by urgency.

WHAT YOU MIGHT BE MISSING — GAPS TO DISCUSS

These are things not in the original app and not in the feature requests, but that will almost certainly become problems once this is in real use.

G1 — Authentication and User Identity (not in scope for MVP, but plan for it)

Right now the app has no login. Anyone with the URL can edit anything. Before going live with a real team, you need at minimum:

- A login page (email + password, or SSO with Google/Azure AD)
- A session/token passed with every API request
- The concept of "the logged-in user" so My Tasks auto-opens to that person's tasks

What to tell your backend developer now: Design the DB and API so that adding auth later does not require a schema change. Specifically: do not hardcode any user identity in the business logic — always pass it as a parameter. When auth is added, it will come from the session, not from the URL.

G2 — Activity log / Audit trail per subtask

Once multiple people use the tracker, you will quickly get the question: "Who changed this? When did this get marked blocked?" The current design has no history.

Minimal version: Add a `subtask_log` table:

sql

```
CREATE TABLE subtask_log (  
  id          INT AUTO_INCREMENT PRIMARY KEY,  
  subtask_id  INT NOT NULL,  
  changed_by  VARCHAR(100),          -- name, until auth is added  
  field_name  VARCHAR(50),          -- e.g. "status", "assignee", "flag_type"  
  old_value   TEXT,  
  new_value   TEXT,  
  changed_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (subtask_id) REFERENCES subtasks(id) ON DELETE CASCADE  
);
```

The backend writes a row here whenever `PUT /api/subtasks/:id` changes `status`, `assignee_id`, `flag_type`, or `flag_reason`. The frontend does not need to do anything for this to work — it just benefits from it when you build a history panel later.

Recommendation: Ask your backend developer to implement this log table and the write-on-update logic now (Phase 0), even if the frontend does not display it yet. Retrofitting audit logging into an existing API is painful.

G3 — Multiple projects per customer

The current app assumes one project per customer. Your data model and the `customers` page only show one project per customer. In practice, a customer like HDFC Bank might have an Implementation project AND a Managed Service project running at the same time.

What needs to change:

- The Customer overview tab should show a list of all projects for that customer, not just one
- `GET /api/customers/:id` should return `projects: []` — all projects linked to that customer
- The customer list sidebar item should show a count of projects, not just one type

This does not block the MVP but it will surface quickly if you have even two projects for one customer.

G4 — Due date on activity groups, not just subtasks

Right now, due dates exist only at the project level and at the individual subtask level. There is no way to say "the Vault Configuration phase should be done by 15 May". This makes it hard to track

milestone-level delays, not just individual steps.

Schema addition (can be added in Phase 0 with no extra cost):

```
sql

ALTER TABLE activity_groups ADD COLUMN due_date DATE;
```

The frontend can then show a phase-level due date in the group header, and the dashboard can flag phases that are overdue as a whole.

G5 — Project progress formula

Currently progress = done subtasks / total subtasks. This treats a 2-minute checkbox the same as a 3-week infrastructure task. Once real data goes in, a project will show 80% progress but still be months from done because the remaining 20% is all the hard work.

Options to discuss now:

1. Keep simple percentage (fine for MVP)
2. Add an `effort_points` or `weight` field to subtasks so progress is weighted
3. Drive progress from phase completion, not individual step completion

No action needed for MVP — but flag this conversation with your team so the formula is a conscious choice, not an accident.

G6 — Browser storage during API transition

While you are building Phase 0 (switching from in-memory to API), you may want a fallback so you can develop the frontend without the backend being ready yet. Consider a thin adapter pattern in your `api.js`:

```
javascript

const USE_API = false; // flip to true when backend is ready

async function getProjects() {
  if (!USE_API) return Promise.resolve(projects); // return in-memory
  const res = await fetch('/api/projects');
  return res.json();
}
```

This lets frontend and backend development proceed in parallel without blocking each other.

G7 — Position / ordering of subtasks and groups

The schema has a `position` column on both `activity_groups` and `subtasks`. The API uses it for ordering. However, the frontend does not yet have drag-to-reorder. This means positions will be assigned on creation (0, 1, 2...) and never change.

This is fine for MVP. Just note that if you add drag-to-reorder later, the backend will need a `PATCH /api/groups/:id/reorder` type endpoint that updates all positions in one transaction — not one-by-one PUT calls.

SUMMARY TABLE — MVP PHASES

Phase	Primary goal	Frontend done when...	Backend needed
0 — Foundation	Data persists on refresh	All existing features work via API, no data loss on reload	Full schema + all base endpoints + seed script
1 — Status + Flag	Explain project delays	Each step has a status badge; blocked steps show in dashboard	<code>/api/dashboard</code> blocked_count + <code>/api/my-tasks</code>
2 — CRUD steps	Customise task lists	Add/edit/delete steps and phases inline	Already covered by Phase 0 endpoints
3 — Assignees + My Tasks	Individual accountability	Each step has an owner; My Tasks page works	<code>/api/my-tasks</code> + team endpoints

Suggested team split:

Backend can start Phase 0 schema and seed immediately. Frontend can run Phase 1 and Phase 2 UI work in parallel (against the in-memory data using the adapter pattern in G6) and then connect to the API when it is ready.

Phases 1 and 2 can be done in parallel by one developer since they do not depend on each other.

Phase 3 depends on Phase 0 (team_members table) being done.

End of document.